

SQL Profiler / EF Core – nasłuch endpointu API

Data: 09.06.2026 17:02

Monitorowany endpoint

GET `http://localhost:5215/api/visits/active`

Narzędzie

EF Core Logging (Microsoft.EntityFrameworkCore.Database.Command) – alternatywa dla SQL Server Profiler zgodnie z wymaganiami projektu.

Opis zapytania

Po wywołaniu endpointu EF Core wysyła SELECT z JOIN-ami: Visits → Patients →AspNetUsers (lekarz) oraz LEFT JOIN VisitMedications. Filtr: aktywne wizyty [Status] IN (0, 1, 2) (Zaplanowana, Potwierdzona, W trakcie) oraz IsDeleted = 0.

Implementacja: VisitsApiController.GetActive → VisitService.GetActiveVisitsAsync().

Wynik wykonania

Executed DbCommand – czas wykonania: 4 ms, baza: ClinicManagerDb.

Screenshot – log EF Core (Executed DbCommand)

Zrzut ekranu z konsoli aplikacji po wywołaniu GET /api/visits/active. Widać pełne zapytanie SQL z JOIN-ami.

```
info: 9.06.2026 16:57:46.652 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (9ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [v].[Id], [v].[Description], [v].[DoctorId], [v].[IsDeleted], [v].[PatientId], [v].[ScheduledDate], [v].[Status], [v].[TotalCost], [p0].[Id], [p0].[FirstName], [p0].[InsuranceNumber], [p0].[IsDeleted], [p0].[LastName], [p0].[Pesel], [p0].[UserId], [a].[Id], [a].[AccessFailedCount], [a].[ConcurrencyStamp], [a].[Email], [a].[EmailConfirmed], [a].[LockoutEnabled], [a].[LockoutEnd], [a].[NormalizedEmail], [a].[NormalizedUserName], [a].[PasswordHash], [a].[PhoneNumber], [a].[PhoneNumberConfirmed], [a].[SecurityStamp], [a].[TwoFactorEnabled], [a].[UserName], [s0].[Id], [s0].[Dosage], [s0].[MedicationId], [s0].[Quantity], [s0].[UnitPrice], [s0].[VisitId], [s0].[Id0], [s0].[Id00]
      FROM [Visits] AS [v]
      INNER JOIN (
        SELECT [p].[Id], [p].[FirstName], [p].[InsuranceNumber], [p].[IsDeleted], [p].[LastName], [p].[Pesel], [p].[UserId]
        FROM [Patients] AS [p]
        WHERE [p].[IsDeleted] = CAST(0 AS bit)
      ) AS [p0] ON [v].[PatientId] = [p0].[Id]
      INNER JOIN [AspNetUsers] AS [a] ON [v].[DoctorId] = [a].[Id]
      LEFT JOIN (
        SELECT [v0].[Id], [v0].[Dosage], [v0].[MedicationId], [v0].[Quantity], [v0].[UnitPrice], [v0].[VisitId], [s].[Id] AS [Id0], [s].[Id0] AS [Id00]
        FROM [VisitMedications] AS [v0]
        INNER JOIN (
          SELECT [v1].[Id], [v1].[IsDeleted], [p2].[Id] AS [Id0], [p2].[IsDeleted] AS [IsDeleted0]
          FROM [Visits] AS [v1]
          INNER JOIN (
            SELECT [p1].[Id], [p1].[IsDeleted]
            FROM [Patients] AS [p1]
            WHERE [p1].[IsDeleted] = CAST(0 AS bit)
          ) AS [p2] ON [v1].[PatientId] = [p2].[Id]
          WHERE [v1].[IsDeleted] = CAST(0 AS bit) AND [p2].[IsDeleted] = CAST(0 AS bit)
        ) AS [s] ON [v0].[VisitId] = [s].[Id]
        WHERE [s].[IsDeleted] = CAST(0 AS bit) AND [s].[IsDeleted0] = CAST(0 AS bit)
      ) AS [s0] ON [v].[Id] = [s0].[VisitId]
      WHERE [v].[IsDeleted] = CAST(0 AS bit) AND [p0].[IsDeleted] = CAST(0 AS bit) AND [v].[IsDeleted] = CAST(0 AS bit) AND [v].[Status] IN (0, 1, 2)
      ORDER BY [v].[ScheduledDate], [v].[Id], [p0].[Id], [a].[Id], [s0].[Id], [s0].[Id0]
```

GET /api/visits/active